Systementwicklungs-Praktikum (SYP)

Vorgaben für Studierende

Autor: **Prof. Dr. René Wörzberger**

gespeichert am: 28.09.2026 12:27

Technology
Arts Sciences
TH Köln

Inhalt

Vorgaben für Studierende....1
Inhalt....2
1 Zweck dieses Dokuments....4
1 Zweck dieses Dokuments....4
2 Zielsetzung Systementwicklungs-Praktikum....5
2 Zielsetzung Systementwicklungs-Praktikum....5
3 Meilensteine....6
3 Meilensteine....6
3.1 Lieferungen....6
3.1.1 Source-Code-Repository....6
3.1.2 Docs-Repository....6
3.1.3 Branching und Tagging....6
3.2 Meilenstein 1 (Anforderungen)....7
3.2.1 Lieferobjekte....7
a) Projektspezifische Anpassungen....7
b) Anforderungen....7
3.2.2 Meilensteinsitzung 1....7
3.3 Meilenstein 2 (Spezifikation und Projektplanung)....7
3.3.1 Lieferobjekte....7
a) Spezifikation....7
b) Projektplanung....7
3.3.2 Meilensteinsitzung 2....7
3.4 Meilenstein 3 (Architektur und Prototyp)....8
3.4.1 Lieferobjekte....8
a) Architektur....8
b) Source-Code Prototyp....8
c) Präsentations-Folien....8
3.4.2 Prototyp....8
3.4.3 Meilensteinsitzung 3 (Präsentation Prototyp)....8
3.5 Meilenstein 4 (Endprodukt)....9
3.5.1 Lieferobjekte....9
a) Source-Code....9
b) Projektdokumentation inkl. Testprotokoll....9
c) Präsentations-Folien....9
3.5.2 Meilensteinsitzung 4 (Präsentation Endprodukt)....9
3.6 Meilenstein 5 (Abnahme)....9
3.6.1 Lieferobjekte....9
a) Betreuer-spezifische Lieferobjekte....9
b) Benutzerdokumentation....9
4 Git und GitLab....11
4 Git und GitLab....11
4.1 Git lokal installieren....11
4.2 Git konfigurieren....11
4.3 In GitLab registrieren....11
4.4 Zugriffsmethoden....12
4.5 SSH-Zugriffsmethode einrichten....12

4.5.1 SSH-Keys erzeugen....12
4.5.2 Public-Key in GitLab-Account hinterlegen....13
4.5.3 gitlab.nt.fh-koeln.de zu den known_hosts hinzufügen....13
4.6 Source-Code-Repository clonen....13
4.7 Lieferung mit Git....13
4.8 Docs-Repository....14
5 Einsatz von Künstlicher Intelligenz....15
5 Einsatz von Künstlicher Intelligenz....15
5.1 Git-Commit-Trailer....15
5.1.1 Trailer in Git-CLI setzen....15
5.1.2 Trailer in Code-Editor setzen....15
5.1.3 Commit-Msg-Hook setzen....16
5.1.4 Trennung zwischen selbstgeschriebenem und generiertem Code....16
5.2 AI-Coding-Agents und -Endpoints....17
5.2.1 Installation von OpenCode....17
5.2.2 Bezug eines API-Keys....17
5.2.3 API-Key in Umgebungsvariable speichern....18
5.2.4 OpenCode konfigurieren....18
5.2.5 Aktuelle THKI-Modellliste abrufen....19
5.2.6 Weiterer Provider....19
a) KISSKI....19
b) AI4S....19
6 Generelle Vorgaben....21
6 Generelle Vorgaben....21

1 Zweck dieses Dokuments

Dieses Dokument erläutert die verbindlichen Vorgaben zum Ablauf und zu Liefergegenständen im Systementwicklungs-Praktikum.

2 Zielsetzung Systementwicklungs-Praktikum

Im Systementwicklungs-Praktikum sollen die Teilnehmer nachweisen, dass sie in einem Projekt ein qualitativ hochwertiges Software-System für eine*n Kund*in erstellen, der/die gleichzeitig auch als Ihr*e Betreuer*in agiert, und dabei die Methoden, Werkzeuge und Hilfsmittel einer systematischen Software-Entwicklung selbständig anwenden können.

Das Systementwicklungs-Praktikum baut somit auf den erworbenen Kenntnissen und Kompetenzen aus den Veranstaltungen „Software Engineering“ und „Software-Praktikum“ auf.

3 Meilensteine

3.1 Lieferungen

Jedes Team erhält Zugang zu einem GitLab-Projekt, über das die Ergebnisse versioniert, ausgetauscht und das Team koordiniert werden kann. Spätestens zu Beginn des Systementwicklungs-Praktikums muss jeder Teilnehmer*in seine oder ihre lokale Git-Installation so konfigurieren und seinen oder ihren GitLab-Zugang so einrichten, wie in Kapitel 4 beschrieben.

Die GitLab-Projekte werden von mir (René Wörzberger) für die Teilnehmer*innen erzeugt und zugeordnet, sobald die Zuordnung von Teilnehmern zu Projekten abgeschlossen wurde. Sie können zwar (zum Ausprobieren) eigene GitLab-Projekte erzeugen, diese sind aber für das SYP irrelevant.

3.1.1 Source-Code-Repository

Jedes GitLab-Projekt hat ein Git-Repository, das im Folgenden "**Source-Code-Repository**" genannt wird. Die Versionierung des Quellcodes, Konfigurationsdateien, Build-Scripts etc. muss über dieses Source-Code-Repository durchgeführt werden.

Das Source-Code-Repository kann über eine URL wie beispielsweise

```
ssh://git@gitlab.nt.fh-koeln.de:10022/sypYY/teamXX.git
```

gecloned werden, wobei die YY durch das jeweilige Kalenderjahr und XX durch Ihre tatsächliche Teamnummer zu ersetzen ist.

(Wenn Sie aus irgendwelchen Gründen einen anderen Git-Repo-Server zur Versionierung bevorzugen, müssen Sie trotzdem Ihren Code bei Lieferungen in das Source-Code-Repository auf `gitlab.nt.fh-koeln.de` pushen. Befassen Sie sich in dem Zusammenhang mit git-remote.)

3.1.2 Docs-Repository

Die kontinuierliche Dokumentation des zu entwickelnden Systems und Projekts erfolgt über das Wiki des jeweiligen GitLab-Projekts. Inhalte des Wikis werden in GitLab wiederum als separates Git-Repository versioniert, im Folgenden "**Docs-Repository**" genannt. Es handelt sich hierbei tatsächlich um ein eigenständiges Git-Repository, das über eine URL wie

```
ssh://git@gitlab.nt.fh-koeln.de:10022/sypYY/teamXX.wiki.git
```

gecloned werden kann.

Weitere Hinweise zur Technik und zu den erwarteten Inhalten finden Sie in der Wiki-Vorlage unter

```
https://gitlab.nt.fh-koeln.de/gitlab/syp/doctemplate/wikis/home
```

Es wird erwartet, dass Sie die Dokumentation dem fortschreitenden Wissensstand im Projekt anpassen. Wenn beispielsweise nach Meilenstein 2 mit dem Betreuer Änderungen an den Anforderungen verabredet werden, sollten die entsprechenden Dateien im Docs-Repository angepasst werden. Zögern Sie nicht, veraltete Informationen zu löschen: Dank Versionierung und Tagging bleiben Zwischenstände für immer erhalten.

3.1.3 Branching und Tagging

Lieferungen für Meilensteine sind im Git-Branch `master` zu liefern und mit einem Git-Tag `MSx` zu versehen. Bitte versehen Sie bei jedem Meilenstein sowohl das Source-Code-Repository als auch das Docs-Repository mit dem gleichen Tag.

Existierende Tags dürfen bis zur jeweiligen Deadline neu gesetzt werden, danach nicht mehr.

Bewertungsrelevant sind nur die durch `MS1`, `MS2`, `MS3`, `MS4` oder `MS5` getaggten Commits. Die Schreibung der Tags (Großbuchstaben MS + exakt eine Ziffer 1 bis 5) ist bindend!

3.2 Meilenstein 1 (Anforderungen)

Zum Termin von Meilenstein 1 müssen Anforderungen beschrieben werden.

3.2.1 Lieferobjekte

a) Projektspezifische Anpassungen

Passen Sie auch Ihren Systemnamen (statt "Parcer") und die Namen und E-Mail-Adressen der Projektmitglieder in der Datei `alldocs.asciidoc` an.

b) Anforderungen

Befüllen Sie im Docs-Repository die Dateien unterhalb von

- `system/01_anforderungen`

Die Dokumentation der Anforderungen orientiert sich an der Lehrveranstaltung "Software-Engineering". Die Anforderungen sollen derart detailliert beschrieben sein, dass die Spezifikation des Systems direkt darauf basieren kann.

3.2.2 Meilensteinsitzung 1

Auf der Sitzung zu Meilenstein 1 werden generelle Auffälligkeiten in den Lieferobjekten besprochen sowie Hinweise zu den Lieferobjekten für Meilenstein 2 gegeben.

3.3 Meilenstein 2 (Spezifikation und Projektplanung)

Zum Termin des zweiten Meilensteins müssen die Systemspezifikation und die Projektplanung abgegeben werden.

3.3.1 Lieferobjekte

a) Spezifikation

Befüllen Sie im Docs-Repository hierzu die Dateien (unterhalb von)

- `system/02_spezifikation`

b) Projektplanung

Befüllen Sie ferner

- `projekt/01_zeitplanung.asciidoc`
- `projekt/02_verantwortungsbereiche.asciidoc`

3.3.2 Meilensteinsitzung 2

Auf der Sitzung zum Meilenstein 2 werden die Lieferungen mit Prof. Wörzberger besprochen. Ggf. wird der Sitzungstermin um Einzeltermine mit ausgewählten Teams ergänzt oder ersetzt.

3.4 Meilenstein 3 (Architektur und Prototyp)

3.4.1 Lieferobjekte

a) Architektur

Zum Termin des dritten Meilensteins müssen im Docs-Repository die Dateien unterhalb von

- `system/03_architektur`

befüllt worden sein.

b) Source-Code Prototyp

Der Source-Code des Prototyps muss vor der Deadline ins Source-Code-Repository gepusht werden.

c) Präsentations-Folien

Die Präsentations-Folien sind unter

- `praesentationen/01_prototyp/Praesentation_Prototyp.pdf`

abzulegen. Abweichend von der Zeitplanung dürfen die Vortragsfolien am Tag vor der Meilensteinsitzung 23:59 Uhr CET geliefert werden.

3.4.2 Prototyp

Es soll ein lauffähiger Prototyp des Systems erstellt werden.

Der genaue Umfang des Prototyps ist abhängig von dem konkret zu erstellenden System.

Der Prototyp sollte interessante und kritische Funktionen enthalten, so dass der/die Kund*in beurteilen kann, ob

- die technische Realisierung der kritischen Anforderungen für das Team möglich ist,
- die Umsetzung der Funktionen in der gewünschten Art und Weise erfolgt.

Implementieren Sie nach Möglichkeit im Prototyp eine oder mehrere einfache, jedoch zentrale Anforderungen als "vertikalen technischen Durchstich" mit zugehörigen Implementierungen in allen architektonischen Schichten (GUI, Fachlogik, Persistenz). Vermeiden Sie umgekehrt bei einer Drei-Schichten-Architektur, dass nur eine Schicht implementiert ist, beispielsweise die Fachlogik und die anderen allein durch Unit-Tests (statt GUI) und Dummies (statt echter Persistenz) realisiert sind. Ein solcher Stand ist selten gegenüber Kunden präsentabel.

3.4.3 Meilensteinsitzung 3 (Präsentation Prototyp)

Das Ziel der Präsentation zum Meilenstein 3 ist die Information des/der Kund*in über den aktuellen Stand des Projekts. Innerhalb der Präsentation muss der Prototyp im aktuellen Stand vorgeführt werden. Der weitere Inhalt und die Form der Präsentation können vom Team in Absprache mit der jeweiligen Dozentin in "Präsentation und Kommunikation" (PuK) bestimmt werden.

Ein Beamer, der über VGA, HDMI, Mini DisplayPort oder USB-C / Thunderbolt 3 angesprochen werden kann, wird gestellt. Ein Presenter kann auch beigestellt werden, wobei das Mitbringen eines eigenen Presenters empfohlen wird. Jedes Team muss das Präsentationsnotebook selbst mitbringen. Bitte melden Sie sich frühzeitig, falls das nicht möglich ist.

3.5 Meilenstein 4 (Endprodukt)

Zum Termin des vierten Meilensteins müssen die Implementierung, der Test und die Dokumentation abgeschlossen sein und abgegeben werden.

3.5.1 Lieferobjekte

a) Source-Code

Der Source-Code des Systems ist erneut in das Source-Code-Repository zu pushen.

b) Projektdokumentation inkl. Testprotokoll

Die Dokumentation im Docs-Repository muss auf einen aktuellen Stand gebracht werden. Insbesondere sind die Dateien unterhalb von

- `projekt inkl.`
- `projekt/03_testprotokoll.asciidoc`

zu aktualisieren bzw. zu erstellen.

c) Präsentations-Folien

Die Folien sollen im Docs-Repository

- `praesentationen/02_endprodukt/Praesentation_Endprodukt.pdf`

abgelegt werden.

3.5.2 Meilensteinsitzung 4 (Präsentation Endprodukt)

Das Ziel der Endprodukt-Präsentation ist die Information des/der Kund*in über das erstellte System.

In der Präsentation muss das fertige System vorgeführt werden.

In der Präsentation ist eine Reflexion über das Praktikum gewünscht. Es soll hierbei kurz dargestellt werden, was gut und was schlecht im Praktikum gelaufen ist (bzgl. des Teams, des Systems, des Betreuers, der Praktikums-Organisation, usw.).

3.6 Meilenstein 5 (Abnahme)

Zum fünften Meilenstein soll das fertige System an den Betreuer übergeben werden. Der Betreuer prüft das übergebene System und nimmt es in der übergebenen Form ab. Falls Mängel vorliegen, wird eine Nachbesserung und erneute Übergabe verlangt.

Mit der erfolgreichen Abnahme des Systems durch den Betreuer ist das Systementwicklungs-Praktikum abgeschlossen.

3.6.1 Lieferobjekte

a) Betreuer-spezifische Lieferobjekte

Der Betreuer bestimmt letztlich, wie die Übergabe erfolgen soll und welche Lieferungen sie umfassen soll. Sprechen Sie mit ihr/ihm frühzeitig darüber.

b) Benutzerdokumentation

Insbesondere müssen die Benutzerdokumentationen fertiggestellt werden. Vorlagen finden sich im Docs-Repository unter

- `system/04_benutzer`

4 Git und GitLab

Lieferungen sollen über das Source-Code-Repository geschehen. Sollten Sie noch niemals mit dem Versionsverwaltungssystem Git gearbeitet haben, so finden Sie unter

https://git-scm.com/book/en/v2

die offizielle Dokumentation, sowie zahllose Tutorials im Internet.

4.1 Git lokal installieren

Installieren Sie (sofern noch nicht geschehen) auf Ihrem Rechner den Git-Kommandozeilen-Client von **https://git-scm.com**. Die Default-Optionen der Installations-Routine können beibehalten werden.

Öffnen Sie danach eine Bash. (Unter Windows sollte "Git Bash" nun installiert sein.)

Führen Sie

```
git --version
```

aus. Die Ausgabe sollte

```
git version 2.16.2.windows.1
```

oder mit einer höheren Versionsnummer oder anderen Plattform lauten.

4.2 Git konfigurieren

Setzen Sie die Git-Properties `user.name` und `user.email` bitte nach folgendem Muster.

```
git config --global user.name "Vorname Nachname"
git config --global user.email "vorname.nachname@smail.th-koeln.de"
```

Wichtig ist vor allem die `user.email`. Verwenden Sie Ihre smail-E-Mail-Adresse für `user.email`. Über diesen Wert werden Commit-Objekte mit in GitLab registrierten Usern korreliert.

Die Option `--global` bewirkt, dass diese Git-Properties als Default für neu erzeugte Repositories auf dem aktuellen Rechner verwendet werden. Sie können diese Properties auch individuell pro lokalem Repository setzen, indem Sie dort obige Zeilen ohne `--global` aufrufen. Das ist dann praktisch, wenn Sie am gleichen Rechner z.B. auch Repositories Ihres evtl. Arbeitgebers ausgecheckt haben, bei dem Sie eine andere E-Mail-Adresse verwenden.

Letztlich ist es für eine ordnungsgemäße Bewertung Ihrer individuellen Leistung wichtig, dass Commits mit der richtigen `user.email` wie oben beschrieben durchgeführt werden. Es können nur Commits in das Source-Code-Repository gepushed werden, wenn diese mit `user.email` wie oben angegeben erzeugt wurden. (Eine nachträgliche Änderung des Autors von lokalen Commits ist möglich, aber sehr aufwändig.)

Sorgen Sie daher im eigenen Interesse dafür. Committen Sie eigene Beiträge auch immer unter eigenem Namen, d.h. mit korrekt gesetzter `user.email`.

4.3 In GitLab registrieren

Jedes Team-Mitglied benötigt einen Account in der GitLab-Instanz, die aus dem Hochschulnetz und über VPN oder Eduroam über

```
https://gitlab.nt.fh-koeln.de/gitlab/
```

erreichbar ist. Verwenden Sie bitte als Username wieder Ihre **kleingeschriebene CampusID** – nicht die Matrikelnummer!! – und als E-Mail-Adresse wieder Ihre smail-E-Mail-Adresse `vorname.nachname@smail.th-koeln.de`. Falls Sie sich bereits in einer vorhergehenden Lehrveranstaltung, z.B. dem SWP, in dieser Instanz mit Ihrer CampusID registriert haben, können Sie den betreffenden Account natürlich weiterverwenden.

Nach dem Login navigieren Sie zu https://gitlab.nt.fh-koeln.de/gitlab/-/user_settings/profile und geben Sie unter `Full Name` Ihren Vor- und Nachnamen an und **stellen Sie sicher, dass in der `Commit email` Ihre smail-E-Mail-Adresse steht**.

Sie sind nach der Registrierung zwar in der Lage, in GitLab eigene Projekte ("unterhalb" Ihres Usernames) anzulegen. Diese sind für das Systementwicklungs-Praktikum aber irrelevant. Nach Ihrer Registrierung werden Ihnen für das für Sie relevante GitLab-Projekt Zugriffsrechte eingeräumt.

Die Verwendung des GitLab-Projekts für Lieferungen ist vorgeschrieben (vorbehaltlich Einwänden des Betreuers). Darüber hinaus ist Ihnen gleichsam freigestellt wie angeraten, die übrige Funktionalität von GitLab zu verwenden. Organisieren Sie die ToDos im Team beispielsweise über Issue-Listen. Es kann aus Kapazitätsgründen jedoch keine Beratung für diese Funktionalitäten geleistet werden. Eine kurze Einführung in GitLab findet z. B. in der Lehrveranstaltung "Software-Management" statt.

4.4 Zugriffsmethoden

Ihr Team erhält bei der Teamzuteilung eine Teamnummer. Angenommen Sie nehmen im Jahr YY teil und haben die Teamnummer XX, dann ist Ihr Source-Code-Repository über http via

```
https://gitlab.nt.fh-koeln.de/gitlab/sypYY/teamXX.git
```

und alternativ mittels SSH über

```
ssh://git@gitlab.nt.fh-koeln.de:10022/sypYY/teamXX.git
```

erreichbar.

Es wird empfohlen, die SSH-Zugriffsmethode zu verwenden. Wenn Sie die https-Methode verwenden, können Sie Abschnitt 4.5 überspringen. Ggf. werden Sie aber dann häufiger nach dem Username und Password gefragt, die Sie beim Registrieren in GitLab vergeben haben.

4.5 SSH-Zugriffsmethode einrichten

4.5.1 SSH-Keys erzeugen

Listen Sie in einer Bash (z.B. GitBash) mittels

```
ls -l ~/.ssh
```

auf, ob das `.ssh`-Verzeichnis existiert und ob darin Private- bzw. Public-Key liegen:

```
~/.ssh/id_rsa
~/.ssh/id_rsa.pub
```

Andernfalls: Führen Sie in der Bash

```
ssh-keygen
```

aus. Dies erzeugt genau die o.g. Dateien an den o.g. Pfaden in der Default-Einstellung. Auf eine Passphrase können Sie verzichten, d.h. die entsprechende Frage mit Return ohne Eingabe beantworten.

Das Schlüsselpaar `id_rsa` (Private-Key) und `id_rsa.pub` (Public-Key) ermöglicht nun, dass ein SSH-Client sich ohne Passworteingabe bei einem SSH-Server authentifiziert. Achtung: Den Private-Key `id_rsa` sollten Sie **niemals** veröffentlichen oder über unsichere Kanäle (E-Mail) versenden. Andernfalls gilt dieser als kompromittiert und unsicher.

4.5.2 Public-Key in GitLab-Account hinterlegen

Hinterlegen Sie in GitLab in `User Settings / SSH Keys` den Inhalt von `~/.ssh/id_rsa.pub` .

Sie können diesen z.B. mittels

```
cat ~/.ssh/id_rsa.pub
```

in einer Bash ausgeben (oder mit z.B. notepad.exe öffnen).

4.5.3 gitlab.nt.fh-koeln.de zu den known_hosts hinzufügen

Umgekehrt muss sich auch ein SSH-Server bei einem SSH-Client authentifizieren. Führen Sie in einer Bash Folgendes aus:

```
ssh-keygen -R [gitlab.nt.fh-koeln.de]:10022
ssh-keyscan -t rsa -p 10022 gitlab.nt.fh-koeln.de >> ~/.ssh/known_hosts
exit
```

Starten Sie die Bash-Shell dann erneut.

4.6 Source-Code-Repository clonen

Clonen Sie Ihr teamspezifisches Repository mittels

```
git clone ssh://git@gitlab.nt.fh-koeln.de:10022/sypYY/teamXX.git
```

wobei Sie YY durch das zweistellige Jahr und XX durch Ihre Teamnummer ersetzen (Achtung: Alle Teamnummern haben zwei Ziffern d.h. ggf. eine führende 0 ggf. ein Suffix a oder b.)

Sie haben nun ein lokales, initial leeres Repository im Verzeichnis teamXX. Hierin können Sie die Projektartefakte wie den Quellcode und die sonstigen Lieferobjekte versionieren und so

- systematisch mit anderen Teammitgliedern teilen sowie
- Lieferobjekte systematisch zur Bewertung liefern

4.7 Lieferung mit Git

Wenn Sie eine zu liefernde Datei erstmalig erstellt haben, so müssen Sie diese mit `git add` zunächst unter Versionskontrolle stellen. Eine typische Lieferung für Meilenstein 3 würde dann so aussehen

```
git commit -am "abschließende Qualitätssicherung der Praesentation"
```

Ein Commit ist quasi ein Snapshot Ihres lokalen Repositories (teamXX), wobei nur die Dateien im Snapshot enthalten sind, die zuvor mit `git add` unter Versionskontrolle gestellt wurden. Die Commit-Message in den Anführungsstrichen ist nur als Beispiel zu verstehen; schreiben Sie hier kurz und prägnant, was seit dem letzten Commit passiert ist.

Sie können beliebig viele Arbeitsstände als Commits verewigen. Über beispielsweise

```
git tag MS1
```

wird der letzte Commit mit dem Tag-Namen `MS1` markiert.

Über

```
git push origin master --tags
```

werden alle Commits (die auf dem `master`-Branch liegen und seit dem letzten Push hinzugekommen sind) in das Source-Code-Repository übertragen neben sämtlichen Tags.

Denken Sie im Interesse Ihres Teams daran, sich genaue Gedanken zu machen, welche Dateien Sie versionieren wollen. Als Faustregel gilt: Nur Dateien, die editiert werden, werden auch versioniert (z.B. `*.java`, `config.json`), generierte Dateien nicht (`*.class`, `*.jar`, `*.log`). Eine gepflegte `.gitignore`-Datei hilft hierbei.

4.8 Docs-Repository

Das Docs-Repository ist als Git-Repository technisch ähnlich zu behandeln wie das Source-Code-Repository. Weitere Details finden Sie unter

```
https://gitlab.nt.fh-koeln.de/gitlab/syp/doctemplate/wikis/home
```

5 Einsatz von Künstlicher Intelligenz

Der Einsatz von (generativer) Künstlicher Intelligenz ist im SYP erlaubt, solange er explizit ausgewiesen wird.

In jedem Fall bleibt jedes Teammitglied gemäß `projekt/02_verantwortungsbereiche.asciidoc` verantwortlich für seine „eigenen“ Dateien unabhängig davon, zu welchen Teilen diese selbst oder mittels KI erstellt wurden. **Jedes Team-Mitglied muss sowohl seinen verantworteten Source-Code als auch die Dokumentation (Asciidoc-Dateien) auf Nachfrage erläutern und kleinere Änderungen daran ohne KI-Unterstützung vornehmen können.**

Setzen Sie KI und insbesondere AI-Coding-Agents mit Bedacht und dosiert ein. Durch reines Vibe-Coding schnell erzielte Fortschritte können zu einer unüberschaubaren Menge unsauberen Source-Codes führen, den dann weder der AI-Coding-Agent selbst noch Sie oder Ihre Team-Mitglieder weiterentwickeln können.

5.1 Git-Commit-Trailer

Zur Dokumentation der KI-Unterstützung ist in jeder Git-Commit-Message anzugeben, ob und wie die aktuell committeten Änderungen maßgeblich mithilfe von generativer KI entstanden sind. Das ist insbesondere dann der Fall, wenn ein AI-Coding-Agent wie OpenCode, Codex oder Claude eingesetzt wurde oder über Prompting in einem AI-Web-Chat wie https://chatgpt.com Code-Blöcke mit mehr als 5 Zeilen generiert wurden.

Dann muss in jeder Git-Commit-Message ein Git-Commit-Trailer im Format

```
Assisted-by: AGENT_NAME:MODEL_VERSION [TOOL1] [TOOL2] ...
```

enthalten sein, beispielsweise

```
Assisted-by: opencode:qwen3.8-27b
```

Wenn die Änderungen eines Commits gänzlich ohne KI-Unterstützung durchgeführt wurden oder die Unterstützung sich auf Autocompletion kleinerer Code-Snippets (bis 5 Zeilen) im Code-Editor beschränkt, kann und soll

```
Assisted-by: none
```

gesetzt werden.

5.1.1 Trailer in Git-CLI setzen

Ein Git-Commit-Trailer lässt sich beispielsweise wie folgt setzen (die \ maskieren die Zeilenumbrüche in der Befehlszeile)

```
git commit \
-m "Die eigentliche Commit-Message ..."  \
--trailer "Assisted-by: opencode:qwen3.8-27b"
```

um zu dokumentieren, dass als AI-Coding-Agent `opencode` verwendet wurde und als Large-Language-Model `qwen3.8-27b`.

5.1.2 Trailer in Code-Editor setzen

Wenn Sie Git statt über die Kommandozeile direkt in z. B. Visual Studio Code oder IntelliJ verwenden, fügen Sie die Trailer-Zeile einfach nach Ihrer eigentlichen Commit-Message ein. Wichtig ist nur, dass Trailer-Zeilen am Ende der Commit-Message eingefügt und durch eine Leerzeile von der eigentlichen Commit-Message getrennt werden.

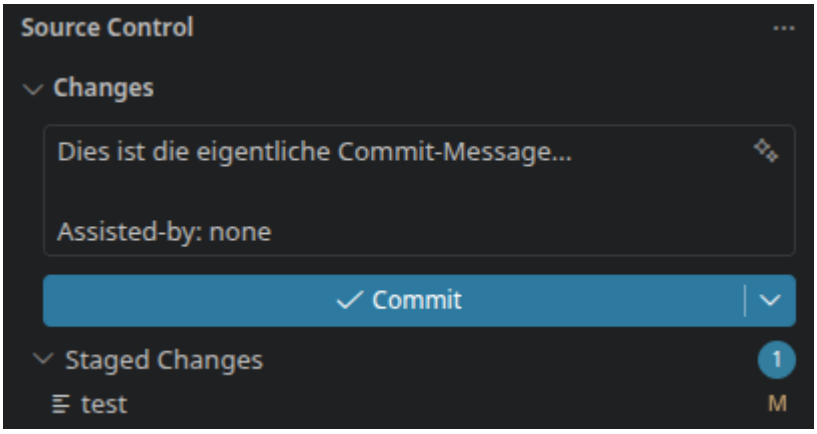


Schaubild 1: Visual Studio Code

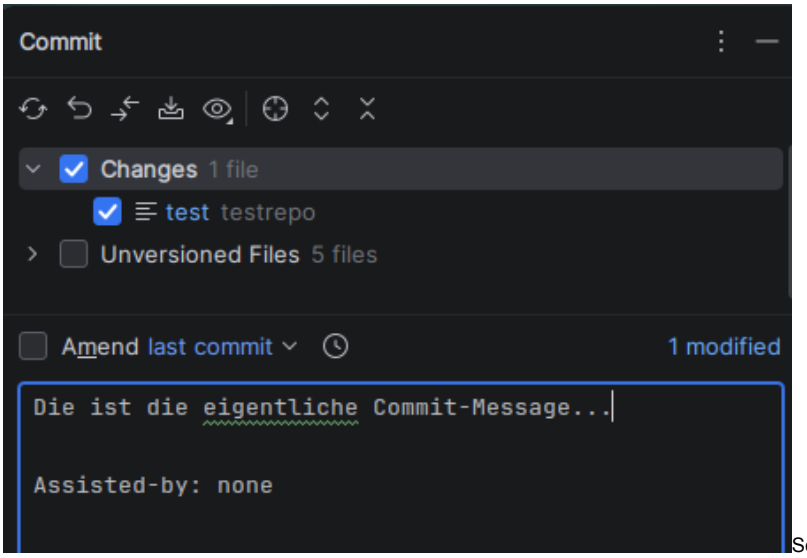


Schaubild 2: IntelliJ IDEA

5.1.3 Commit-Msg-Hook setzen

Damit Sie den Trailer beim Committen nicht vergessen, führen Sie in Ihren lokalen Repositories bitte Folgendes in einer Bash aus:

```
bash <(curl -L https://gitlab.com/rwoerzbe/public/-/raw/main/SYP/git-hooks/install-assisted-by-trailer-hooks.sh)
```

Dies erzeugt zwei neue Hooks in `.git/hooks`, die Commit-Messages auf das Enthaltensein eines `Assisted-by`-Trailers prüfen.

5.1.4 Trennung zwischen selbstgeschriebenem und generiertem Code

Achten Sie darauf, dass Sie generierten Code und händische Nacharbeiten in verschiedenen Commits mit unterschiedlichem `Assisted-by`-Trailer committen. Falls Ihnen zu spät auffällt, dass in Ihrem Worktree bereits beiderlei Änderungen vorhanden sind, informieren Sie sich in der Git-Dokumentation, wie man die Änderungen trotzdem separat committet.

5.2 AI-Coding-Agents und -Endpoints

Die Wahl eines bestimmten AI-Coding-Agents wie Claude, Codex oder OpenCode und des angebundenen Large-Language-Models wie beispielsweise qwen3.8-27b und dessen Provider wie beispielsweise kiconnect.nrw ist Ihnen prinzipiell freigestellt. Falls Sie bislang noch keine eigenen Erfahrungen im Umgang mit der Installation und Konfiguration eines AI-Coding-Agents gesammelt haben, sei Ihnen folgendes Beispiel-Setup an die Hand gegeben.

5.2.1 Installation von OpenCode

Installieren Sie den AI-Coding-Agent OpenCode von https://opencode.ai. OpenCode kann entweder als TUI (Terminal User Interface) betrieben werden oder in einer eigenen Desktop-Anwendung. Die folgenden Ausführungen funktionieren für beide Varianten.

5.2.2 Bezug eines API-Keys

Normale End-User-Hardware verfügt in aller Regel nicht über ausreichend GPU- und VRAM-Kapazitäten, um für das Agentic-Coding geeignete Large-Language-Models darauf lokal zu betreiben. Sie müssen daher OpenCode mit einem externen Model-Provider verbinden.

Als Studierende*r der TH Köln haben Sie Zugriff auf https://chat.kiconnect.nrw/app. Über die Einstellungen links unten gelangen Sie zur API-Schlüsselverwaltung

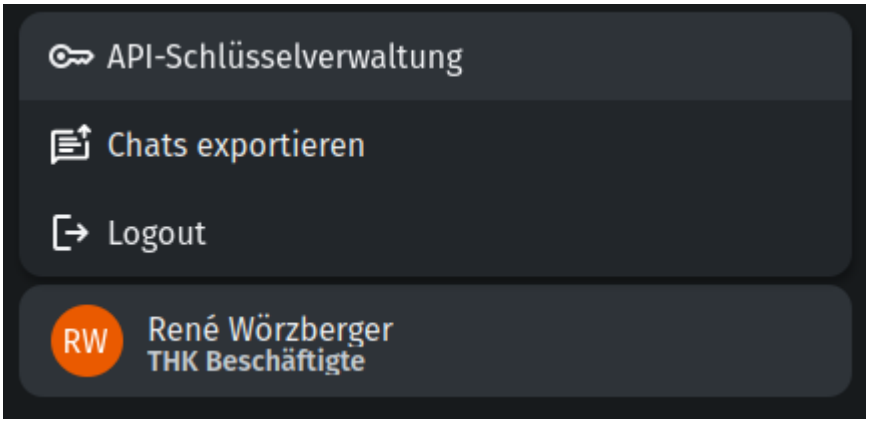


Schaubild 3: THKI-API-Schlüsselverwaltung

Erzeugen Sie dort einen API-Key mit beliebigem Namen und kopieren Sie diesen in die Zwischenablage. (Achtung: Der Schlüssel kann nur direkt nach dem Anlegen kopiert werden; danach nicht mehr.)

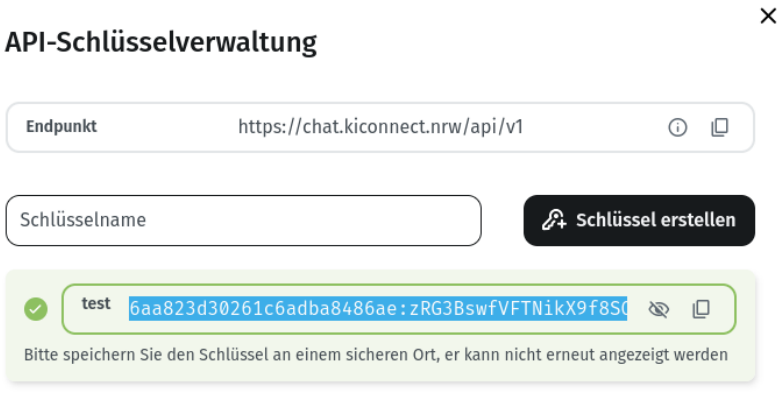


Schaubild 4: neuer Schlüssel

5.2.3 API-Key in Umgebungsvariable speichern

Speichern Sie den Schlüssel in einer Umgebungsvariablen namens `THKI_API_KEY` wie folgt, wobei als Wert für den API-Key im Folgenden beispielhaft `abc:xyz` angenommen wird:

Unter **Windows** öffnen Sie den Ausführungsdialog mit Win+R und führen Sie Folgendes aus:

```
setx THKI_API_KEY "abc:xyz"
```

Unter **Linux** führen Sie Folgendes in einer Bash-Konsole aus:

```
echo 'export THKI_API_KEY="abc:xyz"' >> ~/.bashrc && exit
```

Unter **MacOS** führen Sie Folgendes in einer Zsh-Konsole aus:

```
echo 'export THKI_API_KEY="abc:xyz"' >> ~/.zshrc && exit
```

(Sicherheitshinweis: Unter Linux und MacOS sollte es reichen, wenn nur der aktuelle Benutzer Lese- und Schreibrechte auf der `.bashrc` bzw. `.zshrc` und damit Zugriff auf den API-Key als Secret hat. Stellen Sie dies ggf. mittels

```
chmod 600 ~/.bashrc
```

sicher. Selbstverständlich gibt es noch sicherere Orte als die `.bashrc` bzw. `.zshrc`, Secrets wie API-Keys zu verwahren und nur den autorisierten Prozessen zur Verfügung zu stellen. Deren Erläuterung sprengt hier allerdings den Rahmen.)

5.2.4 OpenCode konfigurieren

Provider, deren API-Endpoints, API-Keys und verfügbare Modelle werden in der Konfigurationsdatei `opencode.json` konfiguriert.

Unter Linux finden Sie diese unter `~/.opencode/opencode.json`

Unter Windows finden Sie diese unter `C:\Users\user\.config\opencode\opencode.jsonc`

Fügen Sie folgenden Abschnitt dort ein:

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "thki": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "thki",
      "options": {
        "baseURL": "https://chat.kiconnect.nrw/api/v1",
        "apiKey": "{env:THKI_API_KEY}"
      },
      "models": {
        "qwen3.5-397b-a17b": {
          "name": "qwen3.5-397b-a17b"
        }
      }
    }
```

```
  }
  "model": "thki/qwen3.5-397b-a17b"
}
```

Starten Sie OpenCode neu.

5.2.5 Aktuelle THKI-Modellliste abrufen

Der konfigurierte Provider THKI aktualisiert die Liste verfügbarer Modelle von Zeit zu Zeit. Sie können die Liste beispielsweise unter Linux oder MacOS mit

```
curl -s https://chat.kiconnect.nrw/api/v1/models \
-H "Authorization: Bearer ${THKI_API_KEY}" |
grep -o '"id":"[^"]*"' |
sed 's/"id":"//;s/"//'
```

abrufen und unter Windows in einer Powershell so

```
(Invoke-RestMethod `
    -Uri "https://chat.kiconnect.nrw/api/v1/models" `
    -Headers @{ Authorization = "Bearer $env:THKI_API_KEY" }).data.id
```

und dann in `opencode.json` übertragen. Sie können die konfigurierten Modelle auswählen, wenn Sie im OpenCode-Eingabefeld `/models` aufrufen.

5.2.6 Weiterer Provider

Der THKI-Provider ist zwar kostenfrei, bietet aber i. d. R. keine ganz aktuellen Modelle an und limitiert Aufrufe, die in einem bestimmten Zeitrahmen stattfinden. Generell kann Ihnen derzeit (2026) kein Modell zur Verfügung gestellt werden, das gleichzeitig topaktuell, unlimitiert und kostenfrei ist.

a) KISSKI

Loggen Sie sich in https://academiccloud.de/ über „Föderierte Anmeldung“ mit Ihrem TH-Köln-Account ein. Navigieren Sie dann zu https://saia.gwdg.de/dashboard und erzeugen dort einen neuen Key, den Sie in der Umgebungsvariablen `KISSKI_API_KEY` speichern.

Fügen Sie dann folgenden Abschnitt innerhalb von `provider` in der `opencode.json` ein:

```
"kisski": {
  "npm": "@ai-sdk/openai-compatible",
  "name": "kisski",
  "options": {
    "baseURL": "https://chat-ai.academiccloud.de/v1",
    "apiKey": "{env:KISSKI_API_KEY}"
  },
  "models": {
    "qwen3.8-27b": {
      "name": "qwen3.8-27b"
    }
  }
}
```

b) AI4S

Das Informatik-Labor stellt über die Forschungsgruppe AI4S ein intern gehostetes Modell zur Verfügung, das allerdings nur im Hochschulnetz (über VPN) erreichbar ist. Folgender Abschnitt muss hierfür innerhalb von `provider` in der `opencode.json` eingefügt werden:

```
"ai4s": {
  "npm": "@ai-sdk/openai-compatible",
  "name": "ai4s",
  "options": {
    "baseURL": "https://llm.vpn.thk-ai4s.de",
    "apiKey": "{env:AI4S_API_KEY}"
  },
  "models": {
    "qwen3.8:27b": {
      "name": "qwen3.8:27b"
     }
   }
}
```

Den Wert für die Umgebungsvariable `AI4S_API_KEY` erhalten Sie auf Anfrage separat.

6 Generelle Vorgaben

- Beachten Sie, wie die **Beispieldokumentation** unter
  https://gitlab.nt.fh-koeln.de/gitlab/syp/doctemplate/wikis
  aufgebaut ist. Die dort gemachten Vorgaben sind mitgeltend.
- Die in **Meilensteinsitzungen** erläuterten Hinweise zur Dokumentation sind ebenfalls mitgeltend (für die folgende Lieferung).
- Prüfen Sie vor einer Meilensteinlieferung, ob der PDF-Export Ihrer Dokumentation korrekt funktioniert.